Actividad del servidor simple al servidor de banco concurrente

Presentado por:
Jessica Lorena Serna Del Rio

Docente:
Jhon James Cano

Cartago Valle
Corporación De Estudios Tecnológicos Del Norte Del Valle
Unidad De Ingeniería, Arquitectura, Urbanismo Y Afines
Ingeniería de Sistemas por Ciclos Propedéuticos
Marzo 03-2026

Introducción

En la actualidad, los sistemas informáticos requieren manejar múltiples usuarios de manera simultánea, garantizando eficiencia, disponibilidad y consistencia en el manejo de datos. La concurrencia es un concepto fundamental en el desarrollo de servidores modernos, ya que permite atender múltiples solicitudes sin bloquear la ejecución del sistema.

En la actividad se desarrolló un servidor asíncrono utilizando el lenguaje de programación Python y la librería asyncio, con el propósito de simular un sistema de atención bancaria concurrente. El ejercicio consistió en transformar un servidor básico en un sistema capaz de:

- Atender múltiples clientes simultáneamente.
- Simular un tiempo de atención real mediante una pausa asíncrona.
- Llevar un conteo global de clientes atendidos.
- Controlar el acceso concurrente a un recurso compartido.
- Medir el impacto del tiempo de respuesta desde el cliente.

Durante el desarrollo se evidenció un problema clásico en sistemas concurrentes: la condición de carrera, la cual surge cuando múltiples procesos acceden y modifican un recurso compartido sin sincronización adecuada. Para resolver esta situación, se implementó un mecanismo de exclusión mutua mediante asyncio.Lock(), garantizando la consistencia del contador global.

La actividad fue desarrollada en un entorno Linux ejecutado a través de Windows Subsystem for Linux (WSL), lo que permitió trabajar en un contexto similar a un servidor real, facilitando la simulación de múltiples conexiones concurrentes.

Este ejercicio no solo permitió comprender el funcionamiento del modelo asíncrono en Python, sino también analizar las diferencias entre concurrencia basada en hilos y concurrencia cooperativa mediante event loop, así como evaluar el comportamiento del sistema ante múltiples clientes simultáneos.

2. OBJETIVOS

2.1 Objetivo General

Diseñar e implementar un servidor asíncrono que simule un sistema de atención bancaria con control de concurrencia y medición de tiempos de atención.

2.2 Objetivos Específicos

- Implementar un servidor basado en sockets utilizando asyncio.
- Simular atención bancaria mediante retardos controlados.
- Controlar el acceso concurrente a un recurso compartido.
- Analizar el comportamiento del sistema bajo múltiples conexiones.

3. DESCRIPCIÓN DEL DESARROLLO

El servidor fue desarrollado en Python 3 utilizando programación asíncrona con asyncio.

Se implementaron los siguientes componentes:

- Socket TCP para comunicación cliente-servidor.
- asyncio.Semaphore() para limitar el número de clientes atendidos simultáneamente.
- Medición de tiempo con el módulo time.
- Simulación de atención bancaria con asyncio.sleep().

Codigo Utilizado
Server_asyncio.py

```
import asyncio
import time

# Recurso compartido
contador_clientes = 0

# Lock para controlar acceso concurrente
lock = asyncio.Lock()

# Semáforo para simular número limitado de cajeros
cajeros = asyncio.Semaphore(2)  # Solo 2 clientes pueden ser atendidos al tiempo
```

```python
async def handle_client(reader, writer):
    global contador_clientes

    addr = writer.get_extra_info('peername')
    print(f"Cliente conectado desde {addr}")

    data = await reader.read(1024)
    nombre = data.decode().strip()

    tiempo_inicio = time.time()

    # Espera turno (control de concurrencia)
    async with cajeros:

        # Controla acceso al recurso compartido
        async with lock:
            contador_clientes += 1
            cliente_actual = contador_clientes

        print(f"Atendiendo cliente #{cliente_actual}: {nombre}")

        # Simula atención bancaria (delay)
        await asyncio.sleep(3)

        tiempo_fin = time.time()
        tiempo_total = round(tiempo_fin - tiempo_inicio, 2)

        respuesta = (
            f"Cliente #{cliente_actual} atendido.\n"
            f"Nombre: {nombre}\n"
            f"Tiempo de atención: {tiempo_total} segundos\n"
            f"Total clientes atendidos: {contador_clientes}\n"
        )

        writer.write(respuesta.encode())
        await writer.drain()

    writer.close()
    await writer.wait_closed()

    print(f"Cliente {nombre} desconectado\n")


async def main():
    server = await asyncio.start_server(
```

```
        handle_client, "0.0.0.0", 5000
    )

    print("Servidor bancario async ejecutándose en puerto 5000...")

    async with server:
        await server.serve_forever()


if __name__ == "__main__":
    asyncio.run(main())
```

Cliente_asyncio.py

```
import asyncio
import time

async def main():
    # Abre conexión con el servidor
    reader, writer = await asyncio.open_connection(
        "127.0.0.1", 5000
    )

    # Solicita el nombre al usuario
    name = input("Ingresa tu nombre: ")

    # Guarda el tiempo inicial
    start_time = time.time()

    # Envía el nombre al servidor
    writer.write(name.encode())

    # Asegura que el mensaje se envíe completamente
    await writer.drain()

    # Espera respuesta del servidor
    data = await reader.read(1024)

    # Guarda el tiempo final
    end_time = time.time()

    # Muestra respuesta
    print(data.decode())
```

```
    # Calcula el tiempo total de atención
    print(f"Tiempo de atención: {round(end_time - start_time, 2)} segundos")

    # Cierra la conexión
    writer.close()
    await writer.wait_closed()


if __name__ == "__main__":
    asyncio.run(main())
```

Pruebas realizadas.

```
jessi@Jessica: /mnt/c/WINDOWS/system32/clase3
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://a
.ms/PSWindows

PS C:\WINDOWS\system32> wsl
jessi@Jessica:/mnt/c/WINDOWS/system32$ cd clase3
jessi@Jessica:/mnt/c/WINDOWS/system32/clase3$ wsl
Command 'wsl' not found, but can be installed with:
sudo apt install wsl
jessi@Jessica:/mnt/c/WINDOWS/system32/clase3$ python3 server_async.py
Servidor bancario async ejecutándose en puerto 5000...
Cliente conectado desde ('127.0.0.1', 38084)
Atendiendo cliente #1: paula
Cliente paula desconectado

Cliente conectado desde ('127.0.0.1', 60160)
Atendiendo cliente #2: maria
Cliente maria desconectado

Cliente conectado desde ('127.0.0.1', 60430)
Atendiendo cliente #3: cata
Cliente cata desconectado

Cliente conectado desde ('127.0.0.1', 47902)
Atendiendo cliente #4: caro
Cliente caro desconectado

Cliente conectado desde ('127.0.0.1', 47914)
Atendiendo cliente #5: santiago
Cliente santiago desconectado

Cliente conectado desde ('127.0.0.1', 55748)
Atendiendo cliente #6: paula
Cliente paula desconectado

Cliente conectado desde ('127.0.0.1', 54366)
Atendiendo cliente #7: paulina
Cliente paulina desconectado
```

Ilustración 2Respuesta servidor a conexiones simultaneas

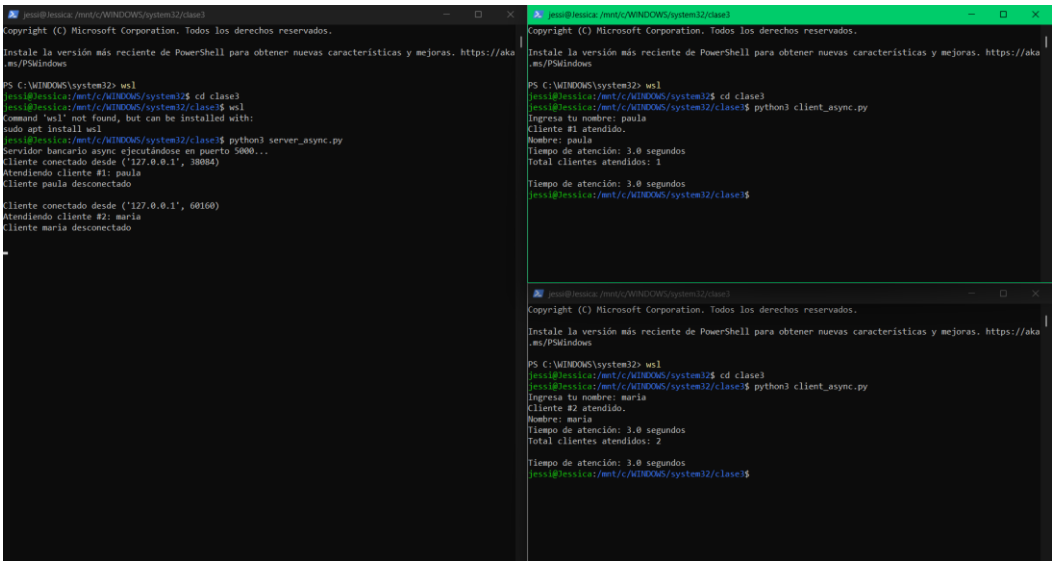

Ilustración 1Peticion cliente hacia el servidor

Cada bloque representa el ciclo completo de atención de un cliente:

1. El cliente establece conexión desde la dirección local 127.0.0.1 (localhost).
2. Se asigna un número consecutivo utilizando el contador global.
3. Se simula el tiempo de atención.
4. El cliente recibe respuesta y se cierra la conexión.

1. ¿Qué diferencia hay entre concurrencia con hilos y async?

La concurrencia con hilos (threads) se basa en la creación de múltiples hilos de ejecución gestionados por el sistema operativo. Cada hilo puede ejecutarse de manera aparentemente simultánea, lo que permite atender varias tareas al mismo tiempo. Sin embargo, este modelo implica un mayor consumo de memoria y costos asociados al cambio de contexto entre hilos.

La concurrencia asíncrona (async) funciona generalmente en un solo hilo utilizando un **event loop** que gestiona múltiples tareas de forma cooperativa. En lugar de ejecutar varias tareas en paralelo real, async permite que una tarea ceda el control mientras espera una operación de entrada/salida (I/O), permitiendo que otra tarea avance.

- **Hilos:** concurrencia basada en paralelismo del sistema operativo.
- **Async:** concurrencia cooperativa basada en eventos.
- **Async es más eficiente para tareas I/O**, mientras que los hilos pueden ser más útiles en tareas intensivas de CPU.

2. ¿Dónde apareció el recurso compartido?

El recurso compartido apareció en la variable global:

contador_clientes

Esta variable era accedida y modificada por todas las conexiones entrantes al servidor.

Cada cliente necesitaba incrementar el mismo contador para obtener su número de atención.

Al ser una variable común para múltiples tareas concurrentes, se convirtió en un punto crítico del sistema, ya que podía generar inconsistencias si no se protegía adecuadamente.

3. ¿Por qué el lock es necesario incluso en async?

Aunque async generalmente trabaja en un solo hilo, múltiples tareas pueden ejecutarse de manera intercalada cuando se utilizan instrucciones await.

Si dos tareas acceden al contador casi al mismo tiempo, podría ocurrir una **condición de carrera**, donde ambas lean el mismo valor antes de que se actualice, generando números duplicados o inconsistentes.

El asyncio.Lock() garantiza exclusión mutua en la sección crítica, asegurando que:

- Solo una tarea a la vez modifique el contador.
- El número asignado a cada cliente sea único y correcto.
- Se mantenga la integridad de los datos.

El uso del lock es fundamental para mantener consistencia incluso en modelos asíncronos.

4. ¿Qué pasaría si se tuvieran 1.000 clientes?

Si el servidor estuviera diseñado únicamente con hilos tradicionales, crear 1.000 hilos podría generar:

- Alto consumo de memoria.
- Sobrecarga del sistema.
- Posibles bloqueos o degradación del rendimiento.

En cambio, con el modelo asíncrono:

- No se crean 1.000 hilos físicos.
- El event loop administra eficientemente las tareas.
- El consumo de recursos es mucho menor.
- El sistema escala mejor en operaciones de red (I/O).

Sin embargo, si cada cliente realizara operaciones intensivas de CPU, el modelo async no sería suficiente y sería necesario combinarlo con procesos adicionales o paralelismo real.

En conclusión, para aplicaciones orientadas a red como servidores y APIs, el modelo async es altamente escalable y eficiente, incluso con un número elevado de clientes simultáneos.

Conclusiones

- El desarrollo del laboratorio permitió evidenciar que la programación asíncrona constituye un modelo altamente eficiente para la gestión de múltiples conexiones concurrentes, especialmente en aplicaciones orientadas a red donde predominan las operaciones de entrada y salida (I/O).
- Se demostró que la concurrencia no implica necesariamente paralelismo físico, sino una adecuada administración del tiempo de espera entre tareas. El modelo basado en event loop permite que el sistema continúe ejecutando otras operaciones mientras una tarea se encuentra en estado de espera, optimizando así el uso de recursos.
- La implementación del servidor asíncrono evidenció que es posible atender múltiples clientes simultáneamente sin incrementar proporcionalmente el tiempo total de ejecución del sistema, lo cual confirma la eficiencia del enfoque cooperativo.
- La incorporación de un contador global permitió identificar un escenario real de recurso compartido, lo que facilitó el análisis de problemas clásicos como la condición de carrera.

Bibliografia

Python Software Foundation. (2024). *asyncio — Asynchronous I/O*. Python 3 Documentation.

https://docs.python.org/3/library/asyncio.html

Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation.

https://docs.python.org/3/library/threading.html

Python Software Foundation. (2024). *socket — Low-level networking interface*. Python 3 Documentation.

https://docs.python.org/3/library/socket.html